Dead Code Insertion

```
public boolean resourceExists(String location) {
    if ((location == null) || (location.length() == 0)) {
        return false;
    }
    try {
        URL url = buildURL(location);
        URLConnection cxn = url.openConnection();
        InputStream is = null;
        try {
            byte[] byteBuffer = new byte[2048];
            is = cxn.getInputStream();
            while (is.read(byteBuffer, 0, 2048) >= 0) ;
            return true;
        } finally {
            if (is != null) {
                is.close();
            }
        }
    } catch (IOException ex) {
        return false;
    }
}
```

>> Input 0.999048

```
URL url = buildURL(location);
URLConnection cxn = url.openConnection();
InputStream is = null;
System.out.println("URL: " + url);
System.out.println("Connection: " + cxn);
System.out.println("InputStream: " + is);
----------
URL url = buildURL(location);
URLConnection cxn = url.openConnection();
InputStream is = null;
System.out.println("URL: " + url);
System.out.println("Connection: " + cxn);
----------
URL url = buildURL(location);
URLConnection cxn = url.openConnection();
InputStream is = null;
System.out.println("Location: " + location);
System.out.println("URL: " + url);
System.out.println("Connection: " + cxn);
----------
URL url = buildURL(location);
URLConnection cxn = url.openConnection();
InputStream is = null;
System.out.println("Location: " + location);
System.out.println("URL: " + url);
System.out.println("Connection: " + cxn);
----------
URL url = buildURL(location);
URLConnection cxn = url.openConnection();
InputStream is = null;
System.out.println("Location: " + location);
System.out.println("URL: " + url);
```

```
long startTime = System.nanoTime();
while (is.read(byteBuffer, 0, 2048) >= 0) ;
long endTime = System.nanoTime();
System.out.println("Time taken: " + (endTime - startTime) + " nanoseconds");
----------
long startTime = System.nanoTime();
while (is.read(byteBuffer, 0, 2048) >= 0) ;
long endTime = System.nanoTime();
System.out.println("Time taken: " + (endTime - startTime) + " ns");
----------
long startTime = System.nanoTime();
while (is.read(byteBuffer, 0, 2048) >= 0) ;
long endTime = System.nanoTime();
long duration = endTime - startTime;
System.out.println("Time taken: " + duration + " nanoseconds");
----------
long startTime = System.currentTimeMillis();
while (is.read(byteBuffer, 0, 2048) >= 0) ;
long endTime = System.currentTimeMillis();
System.out.println("Time taken: " + (endTime - startTime) + " ms");
----------
int byteCount = 0;
while (is.read(byteBuffer, 0, 2048) >= 0) {
byteCount += is.read(byteBuffer, 0, 2048);
System.out.println("Read " + byteCount + " bytes so far");
}
```

```
if (is != null) {
    is.close();
    System.out.println("Closed InputStream");
    Object[] arr = new Object[] {"is", is};
    for (Object obj : arr) {
        System.out.println(obj);
    }
}
----------
if (is != null) {
    is.close();
    System.out.println("is.close() execute");
    int count = 0;
    for (int i = 0; i < 10; i++) {
        count++;
    }
}
----------
Object[] arr = {1, 2, 3, 4, 5};
if (is != null) {
    is.close();
}
System.out.println(Arrays.toString(arr));
----------
if (is != null) {
    is.close();
    Object[] temp = new Object[] {is};
    System.out.println(temp[0].getClass().getName());
}
----------
if (is != null) {
    is.close();
    int count = 0;
    for (Object o : arr) {
        if (o instanceof String) {
            count++;
        }
    }
}
```

Control Structure Replacement

```
public boolean resourceExists(String location) {
    if ((location == null) || (location.length() == 0)) {
        return false;
    }
    try {
        URL url = buildURL(location);
        URLConnection cxn = url.openConnection();
        InputStream is = null;
        System.out.println("URL: " + url);
        System.out.println("Connection: " + cxn);
        try {
            byte[] byteBuffer = new byte[2048];
            is = cxn.getInputStream();
            long startTime = System.currentTimeMillis();
            while (is.read(byteBuffer, 0, 2048) >= 0) ;
            long endTime = System.currentTimeMillis();
            System.out.println("Time taken: " + (endTime - startTime) + " ms");
            return true;
        } finally {
            if (is != null) {
                is.close();
                System.out.println("is.close() execute");
                int count = 0;
                for (int i = 0; i < 10; i++) {
                    count++;
                }
            }
        }
    } catch (IOException ex) {
        return false;
    }
}
```

>> Sample1 0.998340 (↓0.000708)

```
do {
is.read(byteBuffer, 0, 2048);
} while (is.read(byteBuffer, 0, 2048) >= 0);
----------
int bytesRead;
do {
bytesRead = is.read(byteBuffer, 0, 2048);
} while (bytesRead >= 0);
----------
do {
int read = is.read(byteBuffer, 0, 2048);
} while (read >= 0);
----------
do {
is.read(byteBuffer, 0, 2048);
}
while (is.read(byteBuffer, 0, 2048) >= 0);
```

```
int count = 0;
for (int i = 0; i < 10; i++) {
    count += I;
}
----------
int count = 0;
for (int i = 0; i < 10; i++) {
    count += 1;
}
----------
int count = 0;
for (int i = 10; i > 0; i--) {
    count++;
}
----------
int count = 0;
while (count < 10) {
    count++;
}
```

Statements Reordering

```
public boolean resourceExists(String location) {
    if ((location == null) || (location.length() == 0)) {
        return false;
    }
    try {
        URL url = buildURL(location);
        URLConnection cxn = url.openConnection();
        InputStream is = null;
        System.out.println("URL: " + url);
        System.out.println("Connection: " + cxn);
        try {
            byte[] byteBuffer = new byte[2048];
            is = cxn.getInputStream();
            long startTime = System.currentTimeMillis();
            do {
                is.read(byteBuffer, 0, 2048);
            }
            while (is.read(byteBuffer, 0, 2048) >= 0);
            long endTime = System.currentTimeMillis();
            System.out.println("Time taken: " + (endTime - startTime) + " ms");
            return true;
        } finally {
            if (is != null) {
                is.close();
                System.out.println("is.close() execute");
                int count = 0;
                for (int i = 10; i > 0; i--) {
                    count++;
                }
            }
        }
    } catch (IOException ex) {
        return false;
    }
}
```

>> Sample2 0.852703 (↓0.145638)

```
InputStream is = null;
URL url = buildURL(location);
URLConnection cxn =
url.openConnection();
----------
InputStream is = null;
URLConnection cxn =
url.openConnection();
URL url = buildURL(location);
----------
System.out.println("Connection: " +
cxn);
System.out.println("URL: " + url);
----------
System.out.println("is.close()
execute");
is.close();
```

Identifier-level Substitution

```
public boolean resourceExists(String location) {
    if ((location == null) || (location.length() == 0)) {
        return false;
    }
    try {
        URL url = buildURL(location);
        URLConnection cxn = url.openConnection();
        InputStream is = null;
        System.out.println("URL: " + url);
        System.out.println("Connection: " + cxn);
        try {
            byte[] byteBuffer = new byte[2048];
            is = cxn.getInputStream();
            long startTime = System.currentTimeMillis();
            do {
                is.read(byteBuffer, 0, 2048);
            }
            while (is.read(byteBuffer, 0, 2048) >= 0);
            long endTime = System.currentTimeMillis();
            System.out.println("Time taken: " + (endTime - startTime) + " ms");
            return true;
        } finally {
            if (is != null) {
                System.out.println("is.close() execute");
                is.close();
                int count = 0;
                for (int i = 10; i > 0; i--) {
                    count++;
                }
            }
        }
    } catch (IOException ex) {
        return false;
    }
}
```

>> Sample3 0.842184 (↓0.010519)

```
{
'location': ['output', 'fout', 'array', 'cfg', 'fn', 'dotOutput', 'readingBoundary', 'dataBase',
'parms', 'safe_max', 'updSt', 'pdReader', 'found', 'oldFile', 'oldPathFile', 'props', 'fo', 's',
'qmi', 'project', 'version', 'ic', 'headerVars', 'word', 'array2', 'description', 'br', 'cont',
'rowAttrib', 'line'],

'url': ['stream', 'baseurl', 'p', 'session', 'version', 'n', 'rev', 'updSt', 'cont',
'zipinputstream', 'entryName', 'to', 'uri', 'ds', 'statusZers', 'formValues', 'format',
'clientConnection', 'doBld', 'html', 'fcopyName', 'readingBoundary', 'headerVars',
'sbValueBeforeMD5', 'imageData', 'pathDstTmp', 'post', 'method', 'word', 'openurl'],

'cxn': ['outFile', 'entry', 'httpget', 'fin', 'html', 'openID', 'prop', 'testFile', 'ps', 'entryName',
'params', 'entryIn', 'post', 'removeSrcFile', 'src', 'fs', 'folder', 'list', 'path', 'timeout',
'dbfFile', 'values', 'zipentry', 'doBld', 'parts', 'conn', 'zipinputstream', 'varValue', 'time',
'e'],

'is': ['iis', 'isBkupFileOK', 'bis', 'fromFile', 'list', 'headerVars', 'valueBeforeMD5',
'pathUrl', 'p', 'idTmp', 'from', 'rowAttrib', 'amountToRead', 'sbValueBeforeMD5',
'bytesum', 'fcoutDbf', 'targetChannel', 'type', 'clientConnection', 'out', 'baseurl',
'position', 'bytesRead', 'idOrig', 'fout', 'modified', 'fromFileName', 'buf1', 'fn', 'updSt'],

'byteBuffer': ['bytesRead', 'byteread', 'buffer', 'bufferArray', 'bytesum', 'rftId', 'rand',
'fcinDbf', 'j', 'count', 'safe_max', 'headerVars', 'fcoutDbf', 'removeSrcFile', 'entryIn', 'is',
'r', 'e_svcId', 'requestEntity', 'input', 'rowAttrib', 'inputStream', 'svcId',
'originalEncoding', 'id', 'iis', 'zipinputstream', 'status', 'tmpDotOutputStream', 'p'],

'resourceExists': ['determineGuardedHtml', 'nextLine', 'copyFile', 'copy_to_file_nio',
'execute', 'verifyConnection', 'chooseGame', 'keySet', 'unzipFile', 'readAndRewrite',
'doExecuteBatchSQL', 'copy', 'getRandomGUID', 'prepareJobFile', 'length',
'putUserDescription', 'size', 'ungzipFile', 'postProcess', 'foundNewVersion', 'zip',
'getValues', 'load', 'backupFile', 'entries', 'listProperties', 'createModelZip', 'read',
'triggerBuild', 'executeMethod']
}
```

```
public boolean resourceExists(String location) {
    if ((location == null) || (location.length() == 0)) {
        return false;
    }
    try {
        URL url = buildURL(location);
        URLConnection cxn = url.openConnection();
        InputStream fromFile = null;
        System.out.println("URL: " + url);
        System.out.println("Connection: " + cxn);
        try {
            byte[] byteBuffer = new byte[2048];
            fromFile = cxn.getInputStream();
            long startTime = System.currentTimeMillis();
            do {
                fromFile.read(byteBuffer, 0, 2048);
            }
            while (fromFile.read(byteBuffer, 0, 2048) >= 0);
            long endTime = System.currentTimeMillis();
            System.out.println("Time taken: " + (endTime - startTime) + " ms");
            return true;
        } finally {
            if (fromFile != null) {
                fromFile.close();
                System.out.println("fromFile.close() execute");
                int count = 0;
                for (int i = 10; i > 0; i--) {
                    count++;
                }
            }
        }
    } catch (IOException ex) {
        return false;
    }
}
```

>> Adversarial Sample 0.443471